

# WHAT IS MAYO?



# **TABLE OF CONTENTS**

1. Introduction
2. Oil and Vinegar Scheme (Its Predecessor)
  - Implementation
3. Scheme Overview
  - Parameter Sets & Implementation
  - Parameter Trade-offs and Practical Effects
  - Implementation Details
  - Performance & Efficiency:
  - Key Management and Verification Flow
    - Key Generation
    - Signing
    - Verification
- Security Analysis
  - Introduction to MAYO's Security Model
  - Mathematical Foundations and Algebraic Hardness
  - Resistance to Timing and Side-Channel Attacks
  - On-the-Fly Data Generation and Fault Attack Mitigation
  - Hardware Parallelization and Resistance to Differential Power Analysis (DPA)
  - Fault Injection and Physical Security Considerations
  - Overall Security Outlook
4. Comparison of MAYO With Other Post-Quantum Cryptographic Candidates
5. Strengths and Weaknesses of the MAYO Variants
6. Conclusion

Report by:

Jude Albert, Mosharroof Hossain, Labib Nafi, Fahim Tanvir

### **Abstract:**

The rapid development in quantum computing seriously threatens the security of classical cryptographic systems, such as RSA, elliptic curve cryptography, and Diffie-Hellman, based on mathematical problems that can be solved by quantum algorithms efficiently. In response, the National Institute of Standards and Technology (NIST) called for a set of global competitions to determine post-quantum cryptographic standards. MAYO is one such candidate: a multivariate quadratic signature scheme leveraging the Oil and Vinegar method. MAYO provides the following new features that together enhance efficiency and resistance against algebraic attacks: seed-based key generation, compact secret keys, smaller public key sizes owing to “whipped maps,” and layered randomization techniques. MAYO operates on small finite fields and allows for fast key generation, low memory, and multiple parameter sets that balance speed and size with security. Its modular design allows for constrained applications like IoT devices, and higher level variants offer robust protection against quantum adversaries. MAYO has been and will continue to be scrutinized in NIST's evaluation process and shows how modern multivariate cryptography evolves to respond to the challenges of a post-quantum era. It hence opens a promising route toward secure digital signatures, safeguarding communication against quantum computers that may threaten current standards in the future.

### **MAYO: Multivariate Scheme**

#### **1.) Introduction**

The computers that we use today (traditional computers) process bits (0s and 1s). However, they are restricted in that they can only handle these bits one by one. There are certain math problems where computers use brute force in order to find solutions. RSA encryption is designed on a simple, yet clever (for now) concept. It's very easy to multiply two large prime numbers. However, it is very difficult to work backwards and find the two prime numbers when the only thing you are given is the product. A perfect example is  $23 \times 29 = 667$ . That is fairly easy. However, if I'm only given the product (667), what two prime numbers multiplied together will give me this product (667)? RSA encryption understands that scaling the number to hundreds or even thousands of digits long would make the problem impossible for the traditional computer to solve. However, this problem is not impossible for a quantum computer. A brute force approach wouldn't be necessary for a quantum computer since it uses qubits instead of traditional bits. Unlike bits which can only be either 0 or 1, qubits can be both 0 and 1 at the same time until they are measured.

This approach allows for quantum computers to find many solution paths in a way that traditional computers aren't able to follow. These states that the quantum computer creates from 0s and 1s are called superposed states. They manipulate these states in such a way that wrong answers cancel each other out and correct answers reinforce each other. You can think of it like hundreds or thousands of people exploring a maze at the same time and the wrong paths just start eliminating themselves. Why is this a problem? We use problems like these in order to create encryptions that take very long to decode so that information can be made private. When a quantum computer is able to eventually handle these problems, a lot of information that we regard as confidential and private will be made known to the public. In the

academic conference paper by Peter Shor titled "Algorithms for Quantum Computation: Discrete Logarithms and Factoring", he recognized that problems could be made smaller by finding the 'section' of certain math functions. A good example would find a pattern that repeats on wallpaper. Once you see that the pattern keeps repeating after 8 inches, you are able to envision the entire wall. This approach for a quantum computer would make a problem (like finding the two prime numbers of a very large number) possible to solve in polynomial time rather than exponential (hours or days rather than hundreds of years). He saw a vulnerability in our current system and posed the idea of revising it.

In 2015, the NSA (U.S. National Security Agency) informed the world of this security problem. They explained how fast quantum computing was improving. In 2019, Google made significant progress by demonstrating how their 53-qubit computer was able to solve a question in 5 minutes that a supercomputer would need 10,000 years to solve. This raised alarms because it showed the power of how potentially fast quantum computers could be. This year, IBM's Quantum Loon was deployed. It is a processor that tests a quantum computer's components for better error correction. This is essentially what will make the quantum computer able to solve more complex problems, making it one step closer to breaking actual cryptography. Also, this year Google Willow was deployed. It was able to solve a problem in 5 minutes that would normally take supercomputers 10 septillion years to solve. RSA-2048 is the encryption standard that most banks and websites use. The prediction is that by the year 2030, quantum computers will be able to crack our internet security. Before this cracking of internet security, it is likely that the world will panic and there will be economic disruption. The MAYO project is one of many quantum-resistant schemes that we will need to adopt so that internet security is not breached.

Although RSA will eventually be breakable, it is just one of cryptographies that Quantum computers will be able to break into. They will also be able to break into Elliptic Curve Cryptography (used in Bitcoin and phones), Diffie-Hellman (used for secure website connections) and digital signatures (used for verifying software and documents). The security of the internet relies heavily on difficult math problems. Unfortunately, these math problems will eventually be solved by quantum computers when they become powerful enough to handle the calculations for breaking the encryption. This is why NIST asked for the world's best cryptographers to submit their quantum-proof encryption algorithms. 82 different teams from 25 countries submitted their entries. After 8 years of testing, NIST has approved 3 schemas. The rest of the 79 submissions have either been removed or are still under evaluation. MAYO is one of the schemas that is still being evaluated for approval.

Although I mentioned the year 2030 being the approximate time that quantum computers will be able to crack the current encryption, attackers have taken measures to prepare for the future. They have already started to gather encrypted information in spite of not being able to decrypt the information. This is all in an effort to decode it in the future (when quantum computers will have the ability to crack the current system in place). This makes the NSA's approach even more essential and mandatory. This is also where NIST is playing a crucial part in the world's safety. NIST stands for the National Institute of Standards and Technology. It was created by the United States Department of Commerce and aims to further creativity and industrial breakthroughs in America. NIST created the standardization process for post-quantum cryptography because of the potential that future attackers could have on society as a whole in relation to quantum computers. They understood that the present encryption standards would eventually become outdated and easily vulnerable to being cracked. NIST sent a memo to scholars worldwide for quantum-proof cryptographic algorithms. The process involved several rounds with each

algorithm being evaluated. The evaluation was strict, involving characteristics like security and performance. The timeline of these events started in 2016, when NIST began the standardization process. In 2017, 25 countries submitted a total of 82 algorithms. In 2019, 82 candidates were reduced to 26. In 2020, these 26 candidates were reduced to only 15. By 2022, four algorithms were chosen for the standardization process after these three rounds of evaluation. By August 2024, NIST released the final versions of the first three Post Quantum Crypto Standards (FIPS 203, FIPS 204 and FIPS 205). NIST has recently announced a fourth round that it is looking at more digital signature schemes, since there are no more to assess from the original submissions.

MAYO is one of the candidates in the NIST competition for post-quantum signature schemes. It is still being tested and put into deliberation by NIST as it is EUF-CMA secure and has many interesting benefits. To expand, MAYO is a multivariate quadratic scheme where it involves a system of quadratic equations, which is easy to compute but hard to invert, making it appropriate and practical for applications under NIST. It is also a variant of a much older scheme, Oil and Vinegar scheme, which itself is worthy of its own explanations.

To understand why MAYO is promising as a post-quantum signature scheme, it helps to look at how its keys are generated in practice. Even though the math behind MAYO can become very complicated, the real process of generating its keys is surprisingly simple and effective. The whole system starts from something very small called a secret seed. This seed is a few dozen bytes long, which is tiny in respect to most private keys in standard cryptography. From this one seed, the entire secret key is built in an anticipated way. That means that any time you use the same seed, you will always get the same private key. This makes the private key very compact and easy to store or transfer, because you only need to keep the seed rather than a large structure.

From this small seed, MAYO creates all the parts of its private key automatically. These parts include the internal changes, the way the variables are split into “oil” and “vinegar,” and the information required for the extra layers called the whipping layers. All of these elements come straight from the seed, so nothing needs to be stored separately. Once the private key is generated, MAYO uses it to create the public key by applying the private structure to create a set of multivariate equations. These equations are then compressed into a smaller format called  $P^*$ , which keeps the size of the public key much lower than it would normally be. This entire design allows MAYO to have very small keys that are fast to generate and easy to move around, which is important for real-world use.

Another important part of MAYO is the idea of “whipping.” Whipping is a technique used to mix the internal parts of the scheme together in several layers so that the final structure looks random and cannot be easily taken apart by an attacker. You can think of it like taking several decks of cards, shuffling them together many times, and making it nearly impossible to figure out the original order. The purpose of this whipping is to hide the mathematical structure inside the scheme so that even powerful attacks cannot easily analyze the equations.

MAYO also works inside a small finite field called  $GF(16)$ . This is just a mathematical space with only 16 possible values. Operating in such a small field allows MAYO to represent information compactly and perform operations quickly. It also helps reduce the memory requirements, which is useful not just for large computers but also for small devices such as smart cards, embedded chips, and Internet-of-Things

devices. When all of these ideas—whipping, mixed modulus, and small field operations—are combined, the result is a system that is lighter and faster than many older multivariate schemes while still maintaining strong security.

Because MAYO is designed for long-term post-quantum security, it comes in several different versions known as variants. Each variant is meant to reach a different security level. Higher-level variants use larger dimensions and more complicated structures to withstand stronger attackers, while lower-level variants are designed to be smaller, faster, and easier to use on devices with limited power. Even though the sizes and parameters change depending on the security level, the basic structure of MAYO stays the same in every variant. They all use the same seed-based key generation, the same whipping layers, and the same general design. The only differences are in how large or complex the system becomes. You do not need to memorize any numbers or tables to understand this—what matters is that the variants simply scale up or down in size depending on how much security is needed.

Finally, one of MAYO's biggest strengths is its performance during key generation. Many multivariate cryptographic systems are known for being slow or heavy when generating keys because they must build large algebraic systems by hand. MAYO avoids this problem entirely by generating everything from a small seed. This makes key generation extremely fast. Even at the highest security levels, MAYO can produce keys quickly and without using much memory. This efficiency makes it very practical for real devices and real applications, especially in a future where all cryptographic systems will need to be quantum-safe.

As the world moves toward a post-quantum era, it becomes clear to see that cryptography is no longer a passive part of digital security—it is an active battleground. Every improvement in quantum hardware has forced governments, researchers, and industry leaders to rethink how data should be protected not just today, but decades into the future. Sensitive information such as medical records, classified government data, and even everyday financial transactions might still be useful 30–40 years from now. That means if attackers store encrypted data today and wait, quantum computers of the future could disclose everything. This situation is sometimes called “harvest now, decrypt later.” It is one of the biggest reasons behind the fast adoption of post-quantum cryptography.

Another challenge facing the move to quantum-resistant encryption is the big scale at which current cryptographic systems are used. Modern encryption supports banking, smartphones, Wi-Fi routers, satellite communication, cloud technologies, and almost every app that manages user accounts. Redesigning all of these systems to quantum-safe alternatives will not happen right away. It will take years for companies and governments to rewrite software, test compatibility, train employees, and verify that the new systems introduce no unintended weaknesses. In many ways, the cryptographic transition is similar to replacing the foundation of a skyscraper while people are still living inside the building—it requires accuracy, planning, and expert-level engineering.

The need for post-quantum cryptography becomes even clearer when looking at how much digital infrastructure has grown in the last ten years. Technologies like blockchain, cryptocurrencies, and smart contracts have created new types of assets that rely almost entirely on cryptographic guarantees. If these systems become vulnerable to quantum attacks, trillions of dollars could be in danger. Bitcoin's signature scheme, for example, is directly exposed to quantum attacks once public keys are disclosed. This means

that any wallet that has been used even once is at risk if quantum computers reach a high enough level of power. A similar issue affects software updates for phones, computers, and IoT devices. If digital signatures are broken, an attacker could replicate updates and take over millions of devices at once.

This is where schemes like MAYO begin to show their significance. A post-quantum signature scheme must not only be protected—it must be very efficient to support the extensive ecosystem of devices that depend on digital authentication. Billions of devices still have limited memory, slow processors, or minimal power budgets. These include smart thermostats, medical implants, vehicle chips, and industrial sensors. A heavy or slow signature scheme cannot work effectively on that scale. MAYO's compact key sizes, efficient signing, and small memory footprint make it a favorable candidate for these constrained environments.

Another interesting perspective lies in the history of multivariate cryptography itself. The field has existed since the 1980s, but many early proposals didn't succeed due to structural weaknesses that attackers eventually discovered. These schemes were often elegant but lacked defenses against algebraic attacks, linearization attacks, or rank-based attacks. MAYO's design shows how the field has developed. By adding randomized whipping layers, using mixed moduli, and employing seed-based generation, it addresses many of the weaknesses that impaired its predecessors. This evolution shows how cryptographers learn from past failures to create more powerful systems for the future.

Even though MAYO is promising, it is important to understand that NIST's evaluation process is intentionally slow and thorough. A single overlooked flaw could compromise global safety for decades. Because of this, NIST conducts comprehensive public analysis, cryptanalytic challenges, implementation reviews, and side-channel resistance tests. A scheme must not only be mathematically secure but also resistant to physical attacks, such as power analysis or timing measurements. Many algorithms that looked strong on paper were eliminated once cryptographers discovered implementation vulnerabilities. This is why becoming a NIST finalist is already a major achievement, even before formal standardization.

Looking ahead, the post-quantum change will reshape the way society views privacy and communication. Consumers may begin to see software updates or device labels showing "quantum-resistant encryption." Governments will have to update regulations, requiring certain industries to adopt approved post-quantum algorithms. Universities will introduce new courses teaching future programmers how to handle larger key sizes, new math structures, and more complicated signature verification methods. In this sense, the quantum revolution is not only technological—it is educational, economic, and societal.

At the end of the day, the goal of post-quantum cryptography is not to get rid of risk entirely but to make sure that digital security remains ahead of likely adversaries. Quantum computers are an incredible scientific accomplishment, but they also symbolize a major turning point in global cybersecurity. Schemes like MAYO provide a path forward, giving researchers the tools to build a safer future before the threats fully materialize.

Understanding how MAYO keeps its keys small, fast, and reliable gives the bigger picture for why it is taken seriously in the NIST competition. The next step after understanding the structure is to look at the actual parameter sets, benchmarks, and performance results, which show how MAYO

compares to other post-quantum signature schemes. At this point, having understood the concepts behind its key generation and design, we are ready to move on to those real performance numbers.

## **2.) Oil and Vinegar Scheme (Its Predecessor)**

One of the proposed methods to deal with this predicament is the oil and vinegar scheme, proposed in 1997 by Jacques Patarin. This is a popular digital signature scheme in y whose security relies on the difficulty of solving a system of quadratic equations. Given the "vinegar variables" w and x, we can solve for the "oil variables" y and z because the complex system of equations simplifies into a manageable 2×2 matrix problem. For example, if w = 5 and x = 6, we substitute these values into the equations to get:

$$M = \begin{pmatrix} w & x & y & z \end{pmatrix} \begin{pmatrix} \begin{matrix} \text{Vinegar} \\ 1 & 2 \\ 2 & 3 \\ 1 & 3 \\ 1 & -2 \end{matrix} & \begin{matrix} \text{Oil} \\ 1 & 1 \\ 3 & -2 \\ 0 & 0 \\ 0 & 0 \end{matrix} \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix}$$

Vinegar variables
Oil variables
Trap door

$$\begin{pmatrix} y \\ z \end{pmatrix} = \begin{pmatrix} 38 & 95 \\ 44 & 52 \end{pmatrix}^{-1} \begin{pmatrix} 81 \\ 47 \end{pmatrix} \pmod{97} \quad (21)$$

Solving this simplified equation, we can find that y = 5 and z = 6. This setup is a good example of a trapdoor function, where one must choose a random set of variables. Then it must be solved for other variables using a hidden structure (like a modular inverse). The full vector [w,x,y,z]/[w, x, y, z] unlocks the system in this case, metaphorically opening a “trap door”.

This system was looked at since its conception, as one of the main glaring drawbacks is that it produces rather large public keys.

$$\mathcal{P}(\mathbf{v} + \mathbf{o}) = \underbrace{\mathcal{P}'(\mathbf{v}, \mathbf{o})}_{\text{Linear in } \mathbf{o}} + \underbrace{\mathcal{P}(\mathbf{o})}_{=0} + \underbrace{\mathcal{P}(\mathbf{v})}_{\text{fixed}} = \mathbf{t}$$

This can lead to risks in encryption and other vulnerabilities such as leading to degradation in device performance and increased computational load. To address this, MAYO was introduced as a modernized variant working with smaller publics factored in. It has the same structure as its predecessor but differs by reducing the dimension of the zero-evaluation space *O*, using *smaller parameters*.

## **Implementation**



The MAYO scheme is a clever variation of the Oil and Vinegar (OV) signature scheme that primarily focuses on achieving a drastically smaller public key size. This reduction is accomplished by choosing the dimension of the secret Oil subspace  $O$  to be intentionally small, which creates a problem for signature generation that is then solved using a technique called "whipping up" a larger public map.

In a standard OV scheme, the public key  $P$  is a system of quadratic equations. Its security hinges on the difficulty of recovering the secret structure, particularly the Oil subspace  $O$ , from  $P$ . MAYO exploits this relationship by setting the dimension of the Oil space,  $o = \dim(O)$ , to be less than the output dimension  $m$ , i.e.,  $o < m$ . Conceptually, the space  $O$  is a "needle" in the total input space (the "haystack"). By making the needle  $O$  smaller, the designers found that the haystack can also be reduced while maintaining the same difficulty for an attacker trying to find  $O$ . This leads to a massive, 14-fold reduction in the public key size compared to compressed OV schemes at the same security level.

The key difference with standard OV arises during the signing process. The signer samples a random Vinegar part  $v$  and must find the Oil part  $o \in O$  such that the signature  $s = v + o$  satisfies  $P(s) = t$ , where  $t$  is the message digest. When  $v$  is fixed, the equation  $P(v + o) = t$  simplifies to a system of  $m$  linear equations in the  $o$  variables (the coordinates of  $o$ ).

- In standard OV, the number of variables ( $o$ ) is typically greater than or equal to the number of equations ( $m$ ), ensuring a high probability of finding a solution.
- In MAYO\*\*, because  $o < m$ , the system is overdetermined (more equations than variables). Solutions are very unlikely to exist, and the standard signature algorithm breaks down.

To fix the broken signing process while retaining the small public key, MAYO publicly constructs a new map,  $P^*$ , called the "whipped up" map. This map expands the input domain  $k$ -fold, from  $F_q^n$  to  $F_q^{kn}$ , allowing for more variables during signing. The map  $P^*$  is defined using the original public map  $P$  and its differential  $P'(x,y) := P(x + y) - P(x) - P(y)$ , along with fixed public matrices  $E_{ij} \in F_q^{m \times m}$ :

$$P^*(x_1, \dots, x_k) := i = I \sum_k E_{ii} P(x_i) + i = I \sum_k j = i + I \sum_k E_{ij} P'(x_i, x_j)$$

The signature process is now carried out on  $P^*$ . The signer samples  $k$  random Vinegar parts ( $v_1, \dots, v_k$ ) and solves for  $k$  Oil parts ( $o_1, \dots, o_k$ )  $\in O_k$  such that the enlarged signature  $s = (v_1 + o_1, \dots, v_k + o_k)$  satisfies  $P^*(s) = t$ . The critical restoration is achieved by choosing the parameter  $k$  such that:

$$\text{Total number of Oil variables } (ko) > \text{Number of equations } (m)$$

Since the total number of variables  $ko$  is now greater than  $m$ , the system is underdetermined, and solutions are found efficiently using the standard Oil and Vinegar approach (treating the  $o_i$  components as the Oil variables). The key difference is the resulting signature  $s$  is now  $k$ -fold larger, resulting in an increase in signature size (e.g., 2-fold increase), which is the trade-off for the vastly reduced public key.

### **3.) Scheme Overview**

As previously discussed, the main problem with UOV was due to large key sizes, specifically the public key. MAYO solved this by introducing a new method to create maps: Whipped Map. This is essentially a structural trick that would allow us to generate the key deterministically from a small seed. What this essentially means is that at any point we can rebuild the full key whenever we need it. This is much more efficient than storing the entire Matrix. The result of this is drastically smaller key sizes, and it won't hurt performance or security. In order to truly understand how MAYO actually operates, we must first understand its core building blocks. The system parameters define how everything really works. These parameters are essentially the genetic code of the scheme. They determine many different attributes, for example their size, structure, and the level of security. When we change them it changes how strong or fast the system is. We have four main parameters:  $n$ ,  $m$ ,  $o$ , and  $k$ .

First let's start off with  $n$  this is the total number of variables. You can think of this as the overall dimension of the problem space. As the name would suggest these variables are divided into two groups: vinegar and oil. The vinegar variable is what helps mix up the system, and the oil variable is what carries the secret structure that is used during the signing process. Therefore,  $n$  essentially sets the total size of this interaction. The more variables that are added means more complexity which also adds more computing during signing and verification.

Next  $m$ , this represents the number of equations. Each equation represents a relationship between the variables, and when you combine all of them together, it forms what we call the public key questions that the verifier will later check. When the size of  $m$  is increased, the more overrefined the system becomes. In simple terms, this means that it's harder for an attacker to solve without knowing the private key. As logic would suggest, then increasing  $m$  generally increases security, but it's not without its faults. This would also make the verification process a bit heavier, meaning more computation power would be needed.

Then we have  $o$ , which is the oil dimension. This is a subset of the total variables. " $o$ " is what determines how many of those are considered "oil" variables. "oil" variables are a crucial part because they are the ones we solve for during the signature process. When you make the oil dimension smaller, it becomes much harder for someone without the private trapdoor to find the correct signatures and vice versa. Therefore,  $o$  directly affects the delicate balance between security and efficiency MAYO is trying to achieve. A smaller  $o$  would give you a stronger resistance to attacks but will most likely make key generations and signing slightly more complex.

Finally, we have  $k$ ; this represents the whipping parameter. This is a unique feature of MAYO.  $k$  can be thought of as the number of layers or components that are combined to produce the final public map which we will later call  $P^*$ . When we adjust  $k$  the designers can essentially 'whip together' multiple base maps in order to form one composite map. This is the core technique that gives MAYO its featured flexibility. When you have a higher  $k$  value, you get stronger diffusion and potentially even higher security, but as usual it comes at the cost of computation.

Now one of MAYO's biggest innovations is that all of this,  $n, m, o, k$  and even the actual public key can be retrieved from a single short random seed. When you look at your typical multivariate schemes,

you will notice that you must explicitly store large matrices full of coefficients, which can in many cases be hundreds of KB or even MB in size. But MAYO completely sidesteps this outdated practice by using a deterministic generation process. For example, think of the seed like a recipe card. Instead of storing the entire meal, we store instructions and ingredients. Therefore, anyone who has the recipe which is that same seed can recreate the exact same meal but, in this case, the exact same key pair.

These design choices are what makes MAYO incredibly compact and scalable. It essentially means that keys are reproducible, consistent, and take up minimal space. Which is a huge advantage for post quantum cryptography where the key sizes often become a major bottleneck for implementation.

### **Parameter Sets & Implementation**

| Parameter set   | MAYO_one | MAYO_two | MAYO_three | MAYO_five |
|-----------------|----------|----------|------------|-----------|
| security level  | 1        | 1        | 3          | 5         |
| secret key size | 24 B     | 24 B     | 32 B       | 40 B      |
| public key size | 1420 B   | 4912 B   | 2986 B     | 5554 B    |
| signature size  | 454 B    | 186 B    | 681 B      | 964 B     |

Security Levels: MAYO\_one and MAYO\_two both target security level 1 which is comparable to around 128-bit classical security. MAYO\_three jumps to level 3 and MAYO\_five reaches level 5 which is the highest NIST category equivalent to 256-bit security. So as we move from left to right at the table the security gets stronger.

Secret Key Sizes: Second the secret key size. One thing that stands out in MAYO is how tiny the secret keys are: MAYO\_one: 24 bytes, MAYO\_two: 24 bytes, MAYO\_three: 32 bytes and MAYO\_five: 40 bytes. Even at the highest security level the secret key is only 40 bytes. That's extremely compact compared to most post quantum signature schemes.

Public Key Sizes: Public key size is where we start to see trade offs: MAYO\_one: 1420 bytes, MAYO\_two: 4912 bytes, MAYO\_three: 2986 bytes and MAYO\_five: 5554 bytes. You can see there is no perfect linear progression. MAYO\_two actually has one of the largest public keys. This is because MAYO\_two changes internal parameters to reduce signature size which increases the public key.

Signature Sizes MAYO\_one: 454 bytes, MAYO\_two: 186 bytes, MAYO\_three: 681 bytes and MAYO\_five: 964 bytes. The important thing to notice here is that MAYO\_two has the smallest signature only 186 bytes which is significantly smaller than the others. This makes MAYO\_two attractive for bandwidth constrained environments even though the public key is larger.

So the takeaway from this is, each MAYO parameter set balances three competing goals: security, public key size, and signature size. You pick up the level depending on whether you want faster signatures, smaller signatures, or stronger security. And overall MAYO is interesting because it keeps its secret keys extremely small across all levels.

## **Parameter Trade-offs and Practical Effects**

Oil dimension (o): In MAYO, o is relatively small compared to classical UOV. This is part of what lets the public key be compact. But smaller o means signing can be more expensive, because you're solving a linear system with fewer oil variables.

Whipping parameter (k): This determines how many “base maps” are composed (“whipped”) to form the final public map. A larger k increases security but also increases the computational cost of signing.

The two Level 1 variants reflect this trade off MAYO1 uses a smaller public key but pays in a larger signature size. MAYO2 uses a larger public key but has a more compact signature. For Level 3 and Level 5 the authors picked balanced parameter sets (MAYO3 and MAYO5) considering both security and performance. These trade offs are not just small differences in numbers they directly influence how MAYO performs in real world systems especially when memory and bandwidth are constrained.

## **Performance & Efficiency:**

| Variant       | KeyGen / Expansion Cost                     | Signing Cost                                         | Signature Size             | Memory Footprint                                                    |
|---------------|---------------------------------------------|------------------------------------------------------|----------------------------|---------------------------------------------------------------------|
| MAYO1         | Minimal overhead (seed expansion)           | Medium (solving system)                              | 454B                       | Low: compact seed + small map                                       |
| MAYO2         | Same seed-based overhead                    | Slightly faster due to different parameter trade-off | 186B                       | Slightly more PK memory, but still efficient                        |
| MAYO3 / MAYO5 | Higher expansion cost (but still efficient) | Signing cost increases with k and o                  | 681B (MAYO3), 964B (MAYO5) | Larger memory footprint, but manageable for many real-world systems |

Here's some performance insight based on both specification data and reported implementations. Key generation is lightweight since most of the “work” is just expanding a seed not generating massive random matrices. Signing is more expensive than just a hash, but far more efficient than older multivariate schemes with large oil spaces. Verification (not shown in this table) is very efficient because it's evaluating a quadratic map rather than solving a system. Memory remains low, especially when using the compact seed + compressed key representation: very practical for embedded systems.

## **Implementation Details**

How exactly is MAYO implemented in a technical and hardware sense and what makes it practical: The secret key is stored very compactly: only 24 to 40 bytes depending on the variant. That's because the full secret key is generated from a small seed, not stored in its expanded form. The public key is also “seed + compressed data” and the compact representation (cpk) is used in storage or transmission and then expanded when needed. Implementations exist in portable C, but also optimized for AVX2 (desktop) and, potentially for embedded platforms this makes MAYO usable across very different

hardware. The use of a deterministic seed-based expansion (for both SK and PK) dramatically reduces memory and storage costs which is especially valuable for IoT / embedded device use cases.

To validate the efficiency of the MAYO cryptographic variants across diverse computing environments the development team pursued a rigorous implementation and testing strategy. The scheme was written in the C programming language and deployed on the Ubuntu operating system which was a deliberate choice that reflects both technical precision and practical accessibility. C was a low level language which enables direct interaction with hardware resources. This minimizes software overhead and ensures that performance measurements reflect true efficiency rather than artifacts of higher level abstractions. Ubuntu meanwhile provided a stable and widely adopted platform for experimentation, allowing results to be replicated and compared across different hardware generations.

The testing campaign spanned multiple classes of processors which highlights the evolution of MAYO's performance over time. On older yet powerful Intel Xeon chips specifically the Haswell (2013) and Skylake (2015) architectures commonly deployed in servers and high-end desktops. Basically, the scheme demonstrated clear benefits from hardware-level optimizations. The use of AVX2, an advanced vector extension instruction set, proved pivotal. By accelerating mathematical operations at the processor level, AVX2 delivered dramatic speed improvements, underscoring the importance of aligning cryptographic design with available hardware accelerators. These findings confirmed that MAYO could achieve competitive performance even on legacy systems when paired with careful optimization.

The analysis did not stop with traditional x86 architectures. Recognizing the growing dominance of ARM-based processors in mobile devices and lightweight laptops, the developers extended their evaluation to this modern landscape. ARM's design philosophy emphasizes energy efficiency without sacrificing computational capability, making it a critical testbed for cryptographic schemes intended for widespread adoption. Among the ARM-based systems tested, Apple's M3 processor (2023) stood out as the most efficient. Across key operations—generation, signing, and verification—the M3 consistently achieved the lowest cycle counts, demonstrating both speed and energy-conscious performance. This result is particularly significant, as it illustrates MAYO's adaptability to cutting-edge architectures that prioritize mobility and battery life.

MAYO's adaptability across hardware tiers makes it a compelling choice for today's diverse technological landscape. Its seamless performance on Apple's cutting-edge processors mirrors the scheme's own evolution, with both pushing the boundaries of speed, efficiency, and forward compatibility. Meanwhile, its efficient operation on ARMv4-based systems, often found in low-power platforms like Arduino boards and legacy ThinkPads, highlights MAYO's suitability for constrained environments where minimal memory and energy usage are critical. For mainstream commercial systems powered by Intel processors, MAYO delivers reliable, optimized performance that aligns with the needs of general-purpose computing. This broad compatibility ensures that whether deployed in high-end mobile devices, embedded systems, or enterprise desktops, MAYO remains a future-ready cryptographic solution capable of scaling with the pace of modern hardware innovation.

## **Key Management and Verification Flow**

### **Key Generation**

The process starts by building the secret trapdoor. A random, secret matrix  $O$  and a secret seed (seed sk) are generated. The public key is then derived from these secrets: hashing the secret seed creates the public seed (seedpk), which acts as a compact blueprint for the public quadratic equation ( $P$ ). The final public key coefficients are computed from the public seed and the secret  $O$  matrix, ensuring the secret key remains small while the public verification system is complex.

### **Signing**

The core challenge in MAYO is that the math is set up to be overdetermined—there are more equations ( $M$ ) than variables ( $O$ ) to solve, which would normally break the signature process. The fix is the "whipped-up map" ( $P^*$ ), which artificially makes the problem solvable by expanding the entire system  $k$ -fold. This ensures the new number of Oil variables ( $k \cdot O$ ) is greater than the number of equations ( $M$ ), making the system underdetermined and solvable. The signer initiates the trapdoor by first hashing the message and a unique salt to get the target challenge  $t$ . They then randomly select  $k$  sets of Vinegar variables ( $v_1, \dots, v_k$ ). Plugging these random values into the secret system instantly linearizes the equations, allowing the signer to quickly solve for the corresponding  $k$  Oil variables ( $o_1, \dots, o_k$ ). The final signature  $s$  is the concatenation of all the chosen and calculated  $v$  and  $o$  pairs, plus the salt.

### **Verification**

Verification is fast and requires no complex solving. The verifier first regenerates the expected target  $t'$  by hashing the message and the salt provided in the signature. They then reconstruct the full public system,  $P^+$ , using the public key seed. The entire signature vector  $s$  is then plugged directly into the public  $P^*$  system, and the quadratic equation is computed to produce an output vector,  $y$ . The signature is declared VALID only if this computed output  $y$  exactly matches the expected target  $t'$ . This check is secure because any attempt to create a valid  $s$  without the secret trapdoor is equivalent to solving the vast, underlying quadratic system.

The entire workflow of MAYO—from key generation to signature verification—is a direct consequence of a single, intentional implementation choice: setting the dimension of the Oil variables ( $o$ ) smaller than the number of equations ( $m$ ). This choice is made purely to achieve the 14-fold reduction in public key size, which is MAYO's main selling point. However, this same choice breaks the standard signing algorithm because it creates an overdetermined linear system (too many equations, not enough unknowns). Therefore, the subsequent steps in the Key Management flow are not optional; they are structural repairs.

The fix, the whipped-up map ( $P^*$ ), acts as the architectural glue that holds the scheme together.  $P^*$  is not an optional security layer; it is a mathematical tool that restores the trapdoor's functionality. By designing the  $P^*$  map to expand the variable space  $k$ -fold, the scheme successfully ensures that the total number of Oil variables ( $k \cdot o$ ) is greater than the number of equations ( $m$ ). This transforms the unsolvable signing problem into an easily solvable, underdetermined linear system. The consequence of this genius fix is the trade-off seen in the final signature: the  $s$  vector must be  $k$ -times larger to carry the

extra variables needed for the trapdoor to work, but the tiny public key is preserved, demonstrating that the entire MAYO flow is a finely tuned system of mathematical checks and balances.

#### **4.)Security Analysis**

##### **Introduction to MAYO's Security Model**

The MAYO cryptographic scheme is a contemporary development in the realm of multivariate quadratic (MQ) public-key cryptography, building upon the established Oil-and-Vinegar (OV) and Unbalanced Oil-and-Vinegar (UOV) frameworks. The security of these systems is based on the computational difficulty of solving systems of quadratic equations in finite fields a type of problem categorized as NP-hard. Specifically, MAYO's design aims to provide both theoretical and practical protection against classical and quantum threats, while ensuring efficiency in key generation, signing, and verification processes.

The core principle that underpins MAYO's security is the employment of structured multivariate polynomials, which remain impractical to invert without the private key. Consequently, the security of the system relies on the challenge faced by an attacker in recovering the concealed transformation or efficiently solving the disguised equations.

In contrast to alternatives based on lattice or code theories, which are founded on distinct mathematical principles, MAYO's strength stems from the algebraic complexity inherent in these nonlinear systems. Nevertheless, security in contemporary cryptography encompasses more than just mathematical difficulty; it also involves the assessment of implementation security, potential physical vulnerabilities, and defenses against side-channel and fault injection attacks. The MAYO initiative recognizes these factors by integrating not only mathematical evaluations but also comprehensive hardware-level defenses and secure design enhancements.

##### **Mathematical Foundations and Algebraic Hardness**

MAYO draws much of its theoretical security from the well-established Oil-and-Vinegar framework, which has endured extensive cryptanalytic analysis over the years. In the OV framework and its unbalanced forms, the public key symbolizes a set of multivariate quadratic equations  $P(x_1, \dots, x_n)$ , whereas the private key enables the efficient inversion of this system through certain algebraic relationships among the variables. The core challenge in recovering the private transformation or in directly solving the quadratic system is the main defense mechanism.

To strengthen this framework further, MAYO incorporates improvements in its internal parameterization by balancing the size of fields, the number of equations, and the interdependence of variables, making it more complex for known attack types. This includes providing resistance against:

- Direct algebraic attacks, which attempt to exploit structural dependencies to simplify the multivariate system;
- Rank-based and MinRank attacks, which aim to reduce the problem to matrix rank computations;
- Linearization and differential attacks, which leverage partial derivatives or structured relations among equations;

- Quantum-assisted attacks, where Grover’s algorithm and similar quantum search methods offer only quadratic, not exponential speedups, leaving the problem computationally infeasible.

The team responsible for the design of MAYO conducts an extensive mathematical evaluation that assesses its projected bit-level security across various parameter sets, demonstrating that the scheme is on par with NIST's post-quantum security standards. The algebraic structure effectively withstands known cryptanalytic methods without depending on unverified heuristic protections, and its security margins are similar to those of well-established MQ schemes like Rainbow and UOV although it offers more efficient representations of keys and signing processes.

#### Resistance to Timing and Side-Channel Attacks

While the theoretical robustness of a cryptographic scheme is defined by its mathematical complexity, practical security also necessitates careful consideration of side-channel threats. The MAYO implementations, both hardware and software, are crafted to minimize information leakage via timing or power analysis.

A fundamental principle that the designers have adopted is secret-independent control flow. This principle guarantees that the execution route of the algorithm is not influenced by secret data, which prevents attackers from inferring private information from timing discrepancies. In the MAYO signing process, nearly all operations including matrix multiplications, random seed expansions, and affine transformations are executed in constant time. The only exception involves solving linear equations  $Ax = y$  through Gaussian elimination, which, in its basic form, requires conditional operations during the pivoting phase.

To address this vulnerability, the designers of MAYO utilize a constant-time Gaussian elimination method, as recommended by earlier studies. This technique employs a fixed control flow for swapping rows and performing pivot operations, ensuring that execution time remains consistent regardless of secret-dependent data. Empirical assessments of this method show it results in only about a 3% increase in latency, which is a reasonable compromise for the significant enhancement in resistance to side-channel attacks.

This design strategy greatly diminishes the risk of timing-related information leakage, one of the most prevalent threats to hardware and embedded cryptographic systems. It also streamlines the formal verification of constant-time characteristics, aiding in secure hardware synthesis and software compilation.

#### On-the-Fly Data Generation and Fault Attack Mitigation

Another important security enhancement in MAYO pertains to the management of random and pseudo-random matrix data utilized during the signing process. The conventional baseline scheme depends on AES-128-CTR mode with a public seed to enlarge the set of  $P_i$  matrices, which are then kept in memory for later computations. Nevertheless, this memory storage step creates a possible attack vector—especially for fault injection and memory inspection attacks.



To mitigate this risk, the MAYO hardware design incorporates real-time data generation, directing the output of the AES-CTR expansion straight into arithmetic logic units working over GF(16) without the need for intermediate memory storage. This enhancement offers several benefits:

1. Reduced memory footprint, which limits opportunities for attackers to perform targeted memory probing or differential fault analysis;
2. Elimination of persistent intermediate states, preventing leakage of partially expanded pseudorandom data;
3. Equivalent functional correctness, since the computation uses the same deterministic AES-CTR stream but without retaining intermediate data.

The designers recognize that fault injections might still impact the AES expansion process itself; however, such vulnerabilities are not exclusive to MAYO, they are typical of most cryptographic systems that utilize deterministic seed expansion. Therefore, although generating data on-the-fly cannot completely eliminate the potential for fault attacks, it does not expand the attack surface compared to conventional architectures. In fact, by not using memory-stored intermediates, it offers a slight improvement in resistance by narrowing the attacker's opportunity for observation.

#### Hardware Parallelization and Resistance to Differential Power Analysis (DPA)

MAYO's focus on hardware design effectively utilizes parallel computation, taking advantage of the inherent parallelism found in finite-field arithmetic and matrix operations. This approach to parallelization serves a dual function: it significantly enhances performance and throughput while also making power analysis attacks more complex.

In conventional software implementations, differential power analysis (DPA) takes advantage of subtle links between power consumption and operations dependent on secret data to extract key information. However, the execution of tasks in parallel introduces power noise, as the simultaneous activation of multiple logic gates across different data routes complicates the correlation of any individual power trace with specific secret operations. As highlighted in the MAYO documentation, this noise complicates DPA style attacks, as power traces become less predictable and more intertwined with concurrent operations.

However, the designers warn that relying solely on parallelization cannot provide complete protection. Skilled attackers can still compile a substantial number of traces to statistically remove noise. For effective resistance, it is advised to explore the use of masking techniques where intermediate values are randomized through algebraic masking or domain separation as a direction for future research. Masking adds an independent random element into sensitive computations, hindering attackers from finding meaningful correlations, even with a large dataset of traces.

Thus, MAYO's architecture sets a solid foundation for side-channel resistance, but further developments must include masking or threshold implementations to achieve the highest levels of hardware security assurance.

#### Fault Injection and Physical Security Considerations

Fault attacks where an adversary intentionally introduces transient errors through manipulation of voltage, clock, or electromagnetic signals represent a significant risk to hardware-based cryptography. MAYO's security assessment explicitly acknowledges this threat and examines how its design responds to these types of attacks.

Due to the on-the-fly computation model, injected faults may interfere with ongoing AES seed expansions or matrix operations, yet they do not directly reveal private information. In contrast to cryptosystems that maintain precomputed secret dependent intermediate results, MAYO's transient architecture stops the lasting corruption of state data. However, implementing additional fault detection strategies, like redundant computations or error-correcting codes, could enhance its defensive capabilities further.

The evaluation in the documents underscores an important difference between theoretical security and practical implementation. While the cryptographic primitives are strong against mathematical attacks, achieving complete physical security requires the integration of defenses against fault and side-channel vulnerabilities. The team rightly recognizes this as an area that continues to require development.

### Overall Security Outlook

From both theoretical and practical viewpoints, MAYO offers a comprehensive and solid cryptographic proposal. Its algebraic basis draws decades of advancements in multivariate public-key cryptography, which instills confidence in its difficult assumptions. Additionally, its focus on practical implementation aspects such as constant-time execution, real-time data handling, and parallel processing enhances its resilience against practical attack methods.

In comparison to other post-quantum signature contenders, like lattice-based approaches (e.g., Dilithium) or code-based alternatives (e.g., Classic McEliece), MAYO is distinctive for its equilibrium between mathematical clarity and implementation effectiveness. The chosen parameters result in compact signatures and reasonable computational demands while still offering strong theoretical security.

However, the full development of the scheme will rely on ongoing examination from the cryptographic community, including formal verification of constant-time characteristics and additional investigation into masking and redundancy techniques.

#### 4.) Comparison of MAYO With Other Post-Quantum Cryptographic Candidates

| Type                                                         | Sec. Lvl | Key Gen.      | Sign        | Verify     |
|--------------------------------------------------------------|----------|---------------|-------------|------------|
| <b>MAYO [BCC<sup>+</sup>23]</b> (default/pre-expanded)       |          |               |             |            |
| MAYO <sub>1</sub>                                            | 1        | 44k/44k       | 218k/165k   | 54k/31k    |
| MAYO <sub>2</sub>                                            | 1        | 86k/86k       | 240k/142k   | 47k/17k    |
| MAYO <sub>3</sub>                                            | 3        | 169k/169k     | 719k/481k   | 206k/131k  |
| MAYO <sub>5</sub>                                            | 5        | 370k/370k     | 1244k/726k  | 401k/221k  |
| <b>Oil and Vinegar [BCH<sup>+</sup>23]</b> (pkc+skc/classic) |          |               |             |            |
| ovIp                                                         | 1        | 2316k/2341k   | 1548k/79k   | 168k/58k   |
| ovIs                                                         | 1        | 3715k/3734k   | 2063k/83k   | 203k/46k   |
| ovIII                                                        | 3        | 13168k/12832k | 8293k/243k  | 679k/197k  |
| ovV                                                          | 5        | 34989k/35792k | 18802k/462k | 1514k/364k |
| <b>TUOV [DGC<sup>+</sup>]</b> (pkc+skc/classic)              |          |               |             |            |
| tuov-IP                                                      | 1        | 3262k/5916k   | 3639k/134k  | 241k/56k   |
| tuov-Is                                                      | 1        | 11797k/29323k | 19607k/130k | 273k/43k   |
| tuov-III                                                     | 3        | 16237k/29815k | 18020k/354k | 1107k/191k |
| tuov-V                                                       | 5        | 38122k/68089k | 40046k/637k | 2947k/359k |
| <b>Dilithium [LDK<sup>+</sup>20]</b>                         |          |               |             |            |
| dilithium2                                                   | 2        | 81k           | 219k        | 79k        |
| dilithium3                                                   | 3        | 137k          | 355k        | 129k       |
| dilithium5                                                   | 5        | 212k          | 420k        | 204k       |
| <b>Falcon [PFH<sup>+</sup>20]</b>                            |          |               |             |            |
| falcon-512                                                   | 1        | 20672k        | 705k        | 135k       |
| falcon-1024                                                  | 5        | 59019k        | 1427k       | 262k       |
| <b>SPHINCS+ [HBD<sup>+</sup>20]</b>                          |          |               |             |            |
| sha256-128f-simple                                           | 1        | 618k          | 14716k      | 1269k      |

New constructions like MAYO\_One and MAYO\_Two seek to address long-standing shortcomings of previous systems like SNOVA and HAWK-512 as multivariate quadratic (MQ) signature schemes continue to advance in the post-quantum landscape. The five fundamental cryptographic dimensions of key size, signature size, computational speed, memory usage, and underlying security assumptions must be compared in order to evaluate these schemes. Particularly for contemporary, resource-constrained environments like IoT devices and embedded systems, knowing how each scheme performs across these metrics gives insight into its practical viability.

Beginning with key size, the MAYO variants distinguish themselves by producing exceptionally compact public keys. One of the defining weaknesses of earlier MQ systems—most notably SNOVA—is their very large public keys, which can exceed tens of kilobytes due to the expansion of many quadratic polynomials. This makes SNOVA significantly less practical for constrained environments such as IoT sensors or embedded hardware platforms, where storage and transmission budgets are strict. HAWK-512 improves on SNOVA in several dimensions but still cannot match the public-key compactness of MAYO\_One and MAYO\_Two. As a result, the MAYO family clearly stands out among the four candidates for public-key efficiency.

Signature size further highlights the advantages of the MAYO variants. While not always the absolute smallest across all MQ constructions, the signature sizes produced by MAYO\_One and MAYO\_Two remain competitive and compressed, offering a balanced trade-off between compactness and verifiability. SNOVA, by contrast, inherits the structural footprint of classical Oil-and-Vinegar systems and therefore yields signatures that are comparatively larger. Meanwhile, HAWK-512's primary disadvantage is its significantly larger signature sizes, which render it unsuitable for

bandwidth-constrained or low-power systems. These characteristics solidify the MAYO variants' position as strong candidates for applications requiring small, frequent, or network-transmitted signatures.

In terms of performance, Mayo\_One and Mayo\_Two both accomplish incredibly quick verification, which is one of the main performance advantages mentioned in the MAYO literature. Verification can be completed in a fraction of the time needed by many other post-quantum signature schemes thanks to their internal algebraic optimizations and reliance on structured transformations. Their signing speed is not as computationally demanding as more complex MQ or lattice-based constructions, but it is still fast enough for practical applications. HAWK-512's larger signatures and expanded algebraic structures increase computational requirements, further limiting its applicability on devices with limited processor resources, while SNOVA, despite its simplicity, does not provide the same high-performance verification. These performance differences reflect deliberate design decisions: while traditional MQ schemes favored simplicity, the MAYO family optimizes verification efficiency as a key design goal.

Memory usage also plays a critical role in modern cryptographic deployment. The MAYO variants benefit from on-the-fly computation strategies that reduce the need to store large matrices or polynomial sets in memory. This is especially important for verifying signatures in embedded environments with tight RAM constraints. SNOVA's large public keys and polynomial expansions translate to higher memory requirements, creating operational overhead that directly impacts embedded performance. HAWK-512's large signatures also increase the memory necessary for processing and verification. Across these considerations, the MAYO schemes consistently demonstrate superior memory efficiency and implementation practicality.

Finally, security assumptions differentiate the four schemes at a deeper theoretical level. SNOVA and HAWK-512 rely on earlier MQ paradigms, which, while extensively studied, have shown structural vulnerabilities in related designs. By contrast, Mayo\_One and Mayo\_Two introduce and rely upon the Whipped MQ assumption, a strengthened and heavily obscured MQ framework designed to resist the algebraic pattern-extraction attacks that compromised several prior multivariate finalists. While promising, the Whipped MQ assumption is new and therefore less studied than the classical MQ assumptions or the lattice-based systems used by NIST finalists. This lack of long-term cryptanalytic scrutiny is one of the few acknowledged weaknesses of the MAYO family.

### **5.) Strengths and Weaknesses of the MAYO Variants**

Across both Mayo\_One and Mayo\_Two, three central strengths recur. First, MAYO offers compact keys, a trait that significantly enhances its deployability in constrained environments and distinguishes it from SNOVA and many classical MQ designs. Second, MAYO delivers very fast verification, making it ideal for applications that must validate signatures frequently or in real time. Third, its flexible parameters provide developers the ability to tune security levels and performance characteristics without overhauling the entire scheme. These strengths align strongly with the goals of modern, practical PQC.

However, MAYO is not without weaknesses. The first is the reliance on the Whipped MQ assumption, which, although designed to overcome historical MQ vulnerabilities, has not yet undergone the extensive global cryptanalytic examination required for high-confidence standardization. This directly

informs the second weakness: MAYO is less studied than other NIST finalists, particularly those based on lattice or hash-based assumptions that have accumulated decades of analysis.

## **6..) Conclusion**

The MAYO cryptographic scheme showcases a comprehensive approach to post-quantum security by combining strong mathematical foundations with careful implementation-related protections. Its ability to withstand classical algebraic, side channel, and timing-based threats highlights the strength of its design principles. Using methods such as constant-time Gaussian elimination, dynamic AES expansion, and parallel arithmetic processes, the MAYO framework not only delivers exceptional performance but also significantly reduces potential leakage opportunities. While no system is entirely free from all forms of attacks, the MAYO design strikes a thoughtful equilibrium between efficiency and robustness. Future improvements, especially the addition of masking techniques and formal validation of side-channel robustness, will further solidify its reputation as a promising candidate in the next generation of secure, quantum-resistant cryptographic standards.

## References

- The Oil and Vinegar Method – William Buchanan  
[https://www.researchgate.net/profile/William-Buchanan-3/publication/344396950\\_The\\_Oil\\_and\\_Vinegar\\_Method/links/5f7066c4a6fdcc00863f7e36/The-Oil-and-Vinegar-Method.pdf](https://www.researchgate.net/profile/William-Buchanan-3/publication/344396950_The_Oil_and_Vinegar_Method/links/5f7066c4a6fdcc00863f7e36/The-Oil-and-Vinegar-Method.pdf)
- MAYO Presentation (NIST PQC Seminar, Sept 2024) NIST PQC Seminar Slides  
<https://csrc.nist.gov/csrc/media/Projects/post-quantum-cryptography/documents/pqc-seminars/presentations/20-mayo-09242024.pdf>
- MAYO Round 2 Specification Official Specs  
<https://pqmayo.org/assets/specs/mayo-round2.pdf>
- MAYO in Open Quantum Safe (liboqs) Algorithm Overview  
<https://openquantumsafe.org/liboqs/algorithms/sig/mayo.html#:~:text=Algorithm%20type%3A%20Digital%20signature%20scheme.%20Main%20cryptographic%20assumption%3A,Kannwischer.%20Specification%20version%3A%20NIST%20Round%202%20%28February%202025%29.>
- ACM Publication on Post-Quantum Signatures  
<https://dl.acm.org/doi/pdf/10.1145/3658644.3690258>
- MAYO: Practical Post-Quantum Signatures from Oil-and-Vinegar Maps – Ward Beullens  
<https://cosicdatabase.esat.kuleuven.be/backend/publications/files/journal/3390>
- SNOVA Specification Document – Round 2  
SNOVA Specification Document - Round 2
- HAWK v1.1 Specification Document – Round 2  
HAWK v1.1 - Specification Document - Round 2
- MAYO Parameter Timing Benchmarks Params & Timing Data  
<https://pqmayo.org/params-times/>
- MAYO Performance in CPU Cycles (AVX2 Optimizations) ResearchGate Figure  
[https://www.researchgate.net/figure/MAYO-performance-in-CPU-cycles-using-AVX2-optimizations-in-comparison-with-other\\_tbl2\\_378951120](https://www.researchgate.net/figure/MAYO-performance-in-CPU-cycles-using-AVX2-optimizations-in-comparison-with-other_tbl2_378951120)